

Averaging Overflow, Security Review

Target: `src/Average.so`, a `uint256` averaging library **Type:** Correctness / integer overflow, demonstrated by formal verification **Method:** Symbolic execution with Halmos over the full `uint256` x `uint256` input space

This review covers a single utility function, `Average.avg`, and the property a caller would expect of it: the average of two numbers lies between them. The property is stated once as a Halmos check and discharged two ways. On `master` the solver returns a concrete counterexample; on `fixed` it proves the property for every input.

M-01: Averaging overflow produces an out-of-bounds result (Medium)

Root cause

avg forms the sum before halving, inside an unchecked block:

```
function avg(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        return (a + b) / 2;
    }
}
```

For inputs whose sum exceeds `type(uint256).max`, `a + b` wraps modulo 2^{256} . The halved result then has no relation to the true mathematical average and can sit entirely outside `[min(a, b), max(a, b)]`. The unchecked block is what makes the defect silent: without it the addition would revert, turning a wrong answer into a denial of the call.

The property

The bound is the specification. Stated as a Halmos check over symbolic inputs:

```
function check_avg_withinBounds(uint256 a, uint256 b) public pure {
    uint256 m = Average.avg(a, b);
    uint256 lo = a < b ? a : b;
    uint256 hi = a < b ? b : a;
    assert(lo <= m && m <= hi);
}
```

Counterexample (master)

Halmos refutes the property in 0.24s with:

[illegible]

These are $0.75 * 2^{256}$ and $0.25 * 2^{256}$. Their sum is exactly 2^{256} , which wraps to 0, so `avg(a, b)` returns 0 while `min(a, b)` is $0.25 * 2^{256}$. The result is below both inputs. Any consumer that treats the average as a bounded quantity, a price midpoint, a liquidation threshold, a median feed, is handed a value the rest of its logic was never written to expect.

Recommendation

Do not form the sum. Order the inputs and add half the gap to the smaller one:

```
function avg(uint256 a, uint256 b) internal pure returns (uint256) {
    (uint256 lo, uint256 hi) = a < b ? (a, b) : (b, a);
    return lo + (hi - lo) / 2;
}
```

`hi - lo` cannot underflow because `lo <= hi`, and `lo + (hi - lo) / 2 <= hi`, so no intermediate value leaves the `uint256` range. On the `fixed` branch Halmos reports [PASS], proving `lo <= m <= hi` for all inputs.

Severity

Medium. The bug is a correctness violation in a utility function rather than a direct loss of funds in isolation. Its downstream impact is set by the caller: a contract that relies on the average being bounded can be driven to a wrong branch or a zero value, which becomes high severity in a pricing or liquidation path and negligible in a purely informational one. Rated as a proven correctness break with context-dependent consequence.

Why a symbolic proof

A fuzzer draws sample inputs and checks them; it can report success while the failing inputs are ones it never generated. The overflow here lives in a thin slice of the 2^{512} input pairs, the ones whose sum crosses 2^{256} . Halmos does not sample. It treats `a` and `b` as symbolic bitvectors and asks an SMT solver whether any assignment violates the assertion. On `master` the answer is yes, with witnesses; on `fixed` the answer is no, which is a proof of the bound across the entire domain. The value of formal verification is precisely that closing statement: not "no failure was observed" but "no failure exists."

Scope and disclaimer

`Average.sol` is intentionally vulnerable demonstration code written to exercise a formal-verification workflow end to end. It is not a production library and not a real client engagement. The single finding above is a genuine correctness bug in this demo, reproduced and then refuted by machine, not an inflated claim.